Experiment 2

Date : 19-01-2026

K-Nearest Neighbours Classifier

Description :

K nearest neighbour algorithm

is a supervised learning technique that classifies a new data point based on

the similarity with the classes. KNN works by selecting ‘k’ number of

neighbours, calculating the Euclidean distance of k number of neighbours,

taking the k nearest neighbours and assigning the rest of the data to the

category for which number of neighbours is maximum.

Algorithm :

Step

1: Select the number K of the neighbours

Step

2: Calculate the Euclidean distance of K number of neighbors

Step

3: Take the K nearest neighbors as per the calculated Euclidean distance.

Step

4: Among these k neighbors, count the number of the data points in each

category.

Step

5: Assign the new data points to that category for which the number of the

neighbor is maximum.

Dataset :
https://archive.ics.uci.edu/ml/machine-learning-databases/balance-scale/balance-scale.data

Dataset Information :

This data set was generated to model psychological experimental results. Each example is classified as having the balance scale tip to the right, tip to the left, or be balanced. The attributes are the left weight, the left distance, the right weight, and the right distance. The correct way to find the class is the greater of (left-distance * left-weight) and (right-distance * right-weight). If they are equal, it is balanced.

No.of instances: 625

No.of attributes: 4

Dataset Information :

1. Class Name: 3 (L, B, R)
2. Left-Weight: 5 (1, 2, 3, 4, 5)
3. Left-Distance: 5 (1, 2, 3, 4, 5)
4. Right-Weight: 5 (1, 2, 3, 4, 5)
5. Right-Distance: 5 (1, 2, 3, 4, 5)

Ex - 2- KNN

Code - LDA:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import
accuracy_score,confusion_matrix,classification_report
import warnings
warnings.filterwarnings('ignore')
from collections import Counter

# Set plotting style
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette("husl")
```

step-0- Load and Explore the Dataset

```
print("=" * 60)
print("KNN CLASSIFIER - BALANCE SCALE DATASET")
print("=" * 60)
```

```
============================================================
KNN CLASSIFIER - BALANCE SCALE DATASET
============================================================
```

```
url =
'https://archive.ics.uci.edu/ml/machine-learning-databases/balance-
scale/balance-scale.data'

col_names = ['Class','Left-Weight', 'Left-Distance', 'Right-Weight',
'Right-Distance']
data = pd.read_csv(url,header=None,names =col_names)

data.head()
```

	Class	Left-Weight	Left-Distance	Right-Weight	Right-Distance
0	B	1	1	1	1
1	R	1	1	1	2
2	R	1	1	1	3
3	R	1	1	1	4
4	R	1	1	1	5

```
print("\n DATASET INFORMATION:")
print("-" * 40)
print(f"Shape of dataset: {data.shape}")
print(f"Number of instances: {data.shape[0]}")
print(f"Number of attributes: {data.shape[1]}")
print(f"\nColumn names: {list(data.columns)}")
```

```
□ DATASET INFORMATION:
----------------------------------------
Shape of dataset: (625, 5)
Number of instances: 625
Number of attributes: 5

Column names: ['Class', 'Left-Weight', 'Left-Distance', 'Right-
Weight', 'Right-Distance']
```

```
print("\n□ SUMMARY STATISTICS:")
print("-" * 40)
print(data.describe())
```

```
□ SUMMARY STATISTICS:
----------------------------------------
       Left-Weight  Left-Distance  Right-Weight  Right-Distance
count   625.000000     625.000000    625.000000      625.000000
mean      3.000000       3.000000      3.000000        3.000000
std       1.415346       1.415346      1.415346        1.415346
min       1.000000       1.000000      1.000000        1.000000
25%       2.000000       2.000000      2.000000        2.000000
50%       3.000000       3.000000      3.000000        3.000000
75%       4.000000       4.000000      4.000000        4.000000
max       5.000000       5.000000      5.000000        5.000000
```

```
# Encode classes: L=0, B=1, R=2
class_mapping = {'L': 0, 'B': 1, 'R': 2}
data['Class_encoded'] = data['Class'].map(class_mapping)

# Prepare features and target
X = data[['Left-Weight', 'Left-Distance', 'Right-Weight', 'Right-
Distance']].values
y = data['Class_encoded'].values

# Split into train and test
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42, stratify=y)

# Standardize features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Training samples: {X_train_scaled.shape[0]}")
print(f"Test samples: {X_test_scaled.shape[0]}")
print(f"Features: {X_train_scaled.shape[1]}")
print()
```

```
Training samples: 437
Test samples: 188
Features: 4
```

Step 1 select K neighbors

```
k = 5
```

Step-2: Calculate the Euclidean distance of K number of neighbors

```
==========================================
```

EUCLIDEAN DISTANCE FUNCTION

```
==========================================
def euclidean_distance(p1,p2) :
  return np.sqrt(np.sum((p1-p2)**2))
```

```
==========================================
```

KNN ALGORITHM - MANUAL IMPLEMENTATION

```
==========================================
def knn_predict_single(test_pt, X_train,y_train, k= 15) :
    """
    Predict class for a single test point using KNN algorithm

    Steps:
    1. Select the number K of the neighbours
    2. Calculate the Euclidean distance of K number of neighbors
    3. Take the K nearest neighbors as per the calculated Euclidean
distance
    4. Among these k neighbors, count the number of the data points in
each category
    5. Assign the new data points to that category for which the
number of the neighbor is maximum
    """

    # step 1 : select the number k of the neighbors
    # k is passed as parameter

    # step 2 : Cal the euclidean dist of all neighbors
    distances = []
    for i ,train_pt in enumerate(X_train) :
        dist = euclidean_distance(test_pt, train_pt)
        distances.append((dist,y_train[i]))

    # step 3: Take the K nearest neighbors
    # sort distances and take first K
    distances.sort(key=lambda x:x[0])
```

```
    k_nearest = distances[:k]

    # Step 4: Count neighbors in each category
    k_nearest_labels = [label for _, label in k_nearest]
    label_counts = Counter(k_nearest_labels)


    # Step 5: Assign to category with maximum neighbors
    predicted_label = max(label_counts, key=label_counts.get)

    return predicted_label, k_nearest, label_counts



def knn_predict_all(X_test, X_train, y_train, k=5):
    """Predict classes for all test points"""
    predictions = []
    all_neighbors = []
    all_counts = []

    for test_point in X_test:
        pred, neighbors, counts = knn_predict_single(test_point,
X_train, y_train, k)
        predictions.append(pred)
        all_neighbors.append(neighbors)
        all_counts.append(counts)

    return np.array(predictions), all_neighbors, all_counts

# ============================================
# MAIN EXECUTION
# ============================================

# Set K value
K = 5
print(f"□ Using K = {K} neighbors")

# Test with first test sample
print("\n" + "="*60)
print("TESTING KNN ALGORITHM ON FIRST TEST SAMPLE")
print("="*60)

test_sample = X_test_scaled[0]
true_label = y_test[0]
true_class_name = list(class_mapping.keys())
[list(class_mapping.values()).index(true_label)]

print(f"Test Sample Features (scaled): {test_sample}")
print(f"True Class: {true_class_name} ({true_label})")
```

```
# Apply KNN algorithm
predicted_label, k_nearest, label_counts = knn_predict_single(
    test_sample, X_train_scaled, y_train, k=K
)

predicted_class_name = list(class_mapping.keys())
[list(class_mapping.values()).index(predicted_label)]

print(f"\n KNN PREDICTION:")
print(f"Predicted Class: {predicted_class_name} ({predicted_label})")

print(f"\n STEP 1 & 2: Selected K={K} and calculated Euclidean
distances")
print(f"STEP 3: Found {K} nearest neighbors:")

print("\n   Distance | Class | Class Name")
print("   " + "-"*30)
for dist, label in k_nearest:
    class_name = list(class_mapping.keys())
[list(class_mapping.values()).index(label)]
    print(f"   {dist:8.4f} | {label:5} | {class_name}")

print(f"\n STEP 4: Count neighbors in each category:")
for label in sorted(label_counts.keys()):
    class_name = list(class_mapping.keys())
[list(class_mapping.values()).index(label)]
    print(f"   Class {class_name} ({label}): {label_counts[label]}
neighbors")

print(f"\n STEP 5: Assign to category with maximum neighbors")
print(f"   Maximum count: {max(label_counts.values())} for class
{predicted_class_name}")
print(f"\n Prediction {'CORRECT' if predicted_label == true_label
else 'WRONG'}!")

# ============================================
# TEST ON ALL SAMPLES
# ============================================
print("\n" + "="*60)
print("TESTING KNN ON ENTIRE TEST SET")
print("="*60)

# Predict for all test samples
predictions, all_neighbors, all_counts = knn_predict_all(
    X_test_scaled, X_train_scaled, y_train, k=K
)

# Calculate accuracy
correct = np.sum(predictions == y_test)
```

```
# Apply KNN algorithm
predicted_label, k_nearest, label_counts = knn_predict_single(
    test_sample, X_train_scaled, y_train, k=K
)

predicted_class_name = list(class_mapping.keys())
[list(class_mapping.values()).index(predicted_label)]

print(f"\n KNN PREDICTION:")
print(f"Predicted Class: {predicted_class_name} ({predicted_label})")

print(f"\n STEP 1 & 2: Selected K={K} and calculated Euclidean
distances")
print(f"STEP 3: Found {K} nearest neighbors:")

print("\n   Distance | Class | Class Name")
print("   " + "-"*30)
for dist, label in k_nearest:
    class_name = list(class_mapping.keys())
[list(class_mapping.values()).index(label)]
    print(f"   {dist:8.4f} | {label:5} | {class_name}")

print(f"\n STEP 4: Count neighbors in each category:")
for label in sorted(label_counts.keys()):
    class_name = list(class_mapping.keys())
[list(class_mapping.values()).index(label)]
    print(f"   Class {class_name} ({label}): {label_counts[label]}
neighbors")

print(f"\n STEP 5: Assign to category with maximum neighbors")
print(f"   Maximum count: {max(label_counts.values())} for class
{predicted_class_name}")
print(f"\n Prediction {'CORRECT' if predicted_label == true_label
else 'WRONG'}!")

# ============================================
# TEST ON ALL SAMPLES
# ============================================
print("\n" + "="*60)
print("TESTING KNN ON ENTIRE TEST SET")
print("="*60)

# Predict for all test samples
predictions, all_neighbors, all_counts = knn_predict_all(
    X_test_scaled, X_train_scaled, y_train, k=K
)

# Calculate accuracy
correct = np.sum(predictions == y_test)
```

```
total = len(y_test)
accuracy = correct / total * 100

print(f"Total test samples: {total}")
print(f"Correct predictions: {correct}")
print(f"Accuracy: {accuracy:.2f}%")

# ============================================
# TEST DIFFERENT K VALUES
# ============================================
print("\n" + "="*60)
print("TESTING DIFFERENT K VALUES")
print("="*60)

k_values = [1, 3, 5, 7, 9, 11, 15]
accuracies = []

for k in k_values:
    predictions, _, _ = knn_predict_all(X_test_scaled, X_train_scaled,
y_train, k=k)
    accuracy = np.mean(predictions == y_test) * 100
    accuracies.append(accuracy)
    print(f"K = {k:2d}: Accuracy = {accuracy:6.2f}%")

# Find best K
best_k = k_values[np.argmax(accuracies)]
best_accuracy = max(accuracies)
print(f"\n Best K value: {best_k} (Accuracy: {best_accuracy:.2f}%)")
```

```
 Using K = 5 neighbors

============================================================
TESTING KNN ALGORITHM ON FIRST TEST SAMPLE
============================================================
Test Sample Features (scaled): [-1.44890439  1.41747561  0.64165064
1.44907855]
True Class: R (2)

 KNN PREDICTION:
Predicted Class: R (2)

 STEP 1 & 2: Selected K=5 and calculated Euclidean distances
STEP 3: Found 5 nearest neighbors:

   Distance | Class | Class Name
   --------------------------------
     0.6923 |     2 | R
     0.7055 |     2 | R
     0.7146 |     2 | R
```

```
     0.7196 |     2 | R
     0.9885 |     2 | R

 STEP 4: Count neighbors in each category:
   Class R (2): 5 neighbors

 STEP 5: Assign to category with maximum neighbors
   Maximum count: 5 for class R

 Prediction CORRECT!

============================================================
TESTING KNN ON ENTIRE TEST SET
============================================================
Total test samples: 188
Correct predictions: 164
Accuracy: 87.23%

============================================================
TESTING DIFFERENT K VALUES
============================================================
K =  1: Accuracy =  81.38%
K =  3: Accuracy =  84.04%
K =  5: Accuracy =  87.23%
K =  7: Accuracy =  88.83%
K =  9: Accuracy =  89.36%
K = 11: Accuracy =  90.43%
K = 15: Accuracy =  90.96%

 Best K value: 15 (Accuracy: 90.96%)
```